Assignment 2 CS170: Introduction to Artificial Intelligence, Dr. Eamonn Keogh

Yiyang Chen
SID X746706
Email: ychen1420@ucr.edu
March 16, 2026

Project 2: Feature Selection with Nearest Neighbor

All important code is original. Unimportant subroutines that are not completely original are…

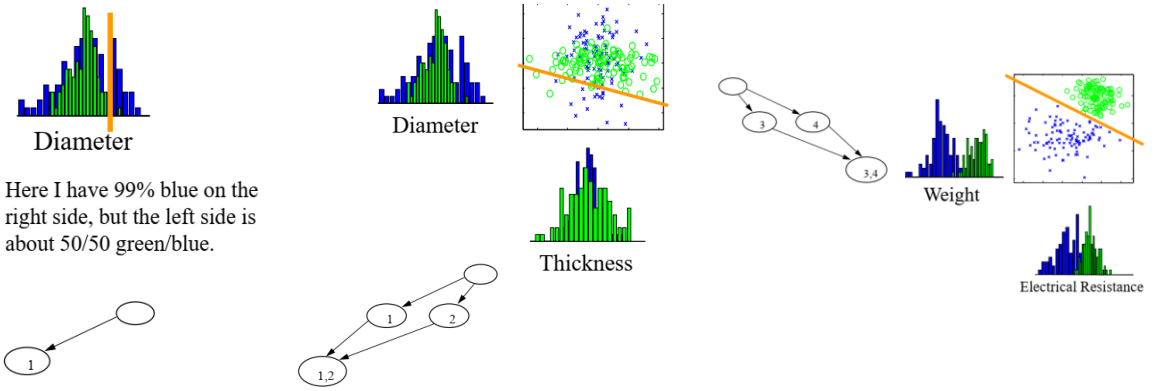
- I used numpy for data loading (np.loadtxt), and array computation in the nearest_neighbor( ) function.
- I imported the time library, and used the time.time( ) function to measure the search time.

Outline of this report:

- Cover page: (this page)
- My report: Pages 2 to 10.
- My code pages 11 to 15. Note that in case you want to run my code, here is a URL that points to it online: https://github.com/cyy-07/UCR-CS170-Project2_ML
- Extra Credit: page 16-17

Introduction

The Nearest Neighbor algorithm, as one of the fundamental ML algorithms we studied in class, works by finding the closest instance in the training data, and assigning its label to the unknown sample. It is simple to understand, and it has no training phase compared to the linear classifier, which needs to learn a decision boundary (because it just stores all the data). However, it needs more computation during the prediction phase, and is quite sensitive to irrelevant features. According to Dr. Keogh's lecture slides[1], there is a good illustration of the importance of feature selection. As shown in Figure 1, in the task of classifying Canadian vs. American quarters, not all features are equally useful. If we only use diameter alone, the algorithm performs poorly, and even combining two uninformative features does not help.



(a) Diameter only (b) Diameter + Thickness (c) Weight + Electrical Resistance

Figure 1(a)(b)(c): Comparison of feature selection (from course lecture slides, Dr. Keogh) [1]

(Note that the decision boundaries shown in Figure 1 are only used for illustration, to demonstrate the "goodness" and importance of feature selection. The actual classifier using the Nearest Neighbor algorithm, will produce Voronoi decision boundaries rather than linear ones.)

In this project, we are required to complete the feature selection in nearest neighbor algorithm, using forward selection and backward elimination. We are given two datasets, small_dataset_1.txt and large_dataset_1.txt[2], with features and instances. Our goal is to identify the best subset of features that maximizes the accuracy of

[1] From Dr. Keogh's lecture slide: 7__MachineLearning002.pdf, page 11-20. The URL is here: https://www.dropbox.com/scl/fo/blbkjaf1eyl94lij5wl2b/AAvNKn0YrnaX0oGPQn7ueFo?rlkey=alq2gb2ftsw73hcar4lk897r0&e=2&dl=0

[2] As an extension student, my name seems not on the list. Dr. Keogh suggests me to use both dataset 1.

classification. Both algorithms use leave-one-out validation, which is a special case of k-fold cross validation and k is equal to the number of instances.

Algorithm Overview

(1) Forward selection. As we learned from the class, if we are given N features, there are 2^N possible feature subsets to evaluate, which needs a significant computation time. Therefore, we resorted to greedy search. As shown in Figure 2, forward selection starts from the empty set (at the top) and greedily moves down, and adds one feature each time by selecting the highest accuracy feature at each level. Finally, it continues until at the full set of features, compares all the best accuracy at each level and outputs the best total accuracy.

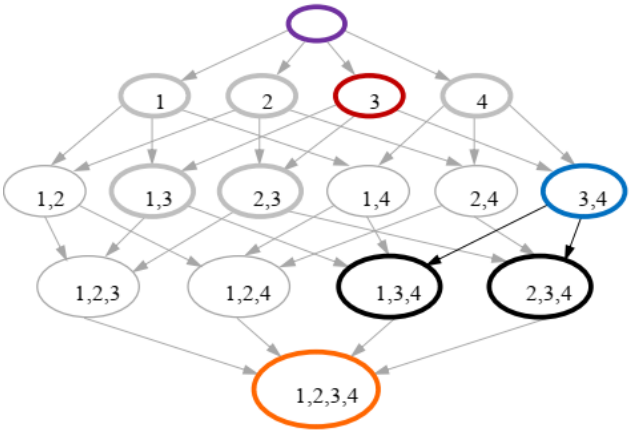


Figure 2: Example search space for forward selection with 4 features (adapted from course lecture slides, Dr. Keogh[3])

(2) Backward elimination. Backward elimination works quite similarly to the forward selection, but starts from the full feature set instead of an empty set, and removes one feature each time at each level. The greedy choice is: we remove the feature that gives the highest accuracy after backward elimination. Finally, it repeats the same process at each level, also compares all the best accuracy at each level and outputs the best total accuracy.

In my project, I implemented the above algorithms in Python. Taking forward selection as an example, at each level, I temporarily add each candidate feature to the current feature set and evaluate the accuracy. The nn_classify( ) function computes the Euclidean distance from each instance to all others using only the

[3] From Dr. Keogh's lecture slide: 7__MachineLearning002.pdf, page 25. The URL is here: https://www.dropbox.com/scl/fo/blbkjaf1eyl94lij5wl2b/AAvNKn0YrnaX0oGPQn7ueFo?rlkey=alq2gb2ftsw73hcar4lk897r0&e=2&dl=0

selected features. For the highest accuracy at each level, the feature is selected and updated in the feature set.

Small Dataset Analysis

The small dataset (dataset number1) contains 500 instances, each belonging to either class 1 or class 2 in the first column, with 16 features. The first column represents the class label, and the remaining columns represent the features. At first, I tried to get the default rate of the small dataset[4]. Surprisingly, class 1 has only 85 instances while class 2 has 415 instances. Therefore, the default rate is 415/500=0.83=83%, which means that if we just simply guess the label as class 2, we still get the accuracy of 83.0%. At the beginning, I also tried the classifier using all 16 features with leave-one-out validation, getting a result of only 70.2%. Interestingly, this is lower than the default rate, which means that even using all 16 features it actually performs worse than simply guessing the majority class every time. I strongly suspect that most of the 16 features are irrelevant noise and I began my search.

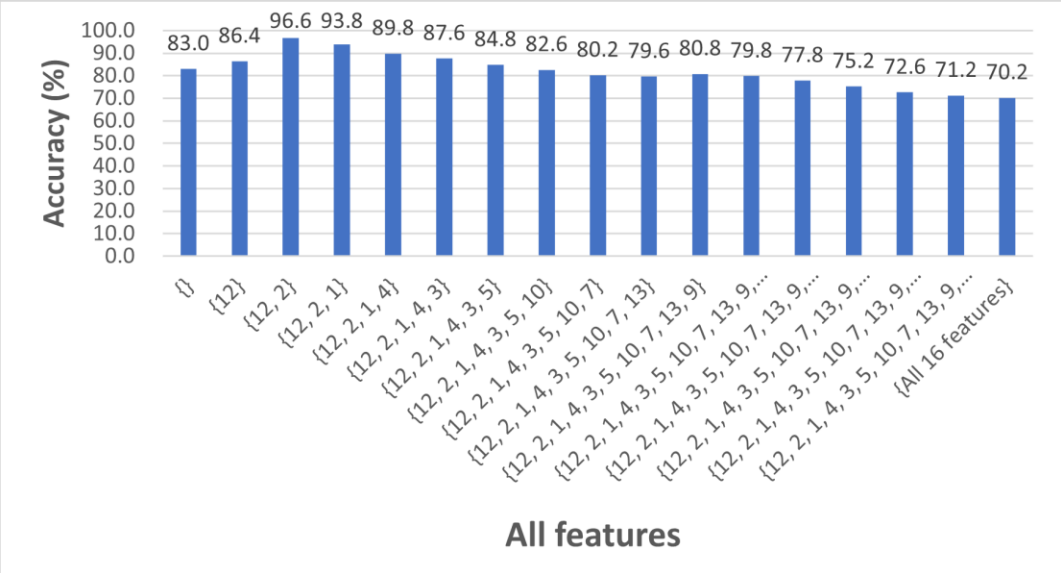


Figure 3: Accuracy changes during forward selection in small dataset 1

In Figure 3, we are able to see the result of running forward selection. Starting from the empty feature set, I expand the search tree, corresponding to the default rate of

[4] It can be implemented by three lines of python: labels = dataset[:, 0]
print(f"Class 1: {int(np.sum(labels==1))}")
print(f"Class 2: {int(np.sum(labels==2))}")

83.0%. At the first layer, adding feature 12 dramatically improved the accuracy to 86.4%, which indicates that this feature can provide useful information in my classification. At the second layer, when the algorithm added feature 2, the accuracy was further pushed to 96.6%. However, when additional features were added, the performance began to decrease. Adding feature 1 at the third layer reduced the accuracy to 93.8%, and adding feature 4 decreased the accuracy to 89.8%. After that, the accuracy kept decreasing in the following layers when more features were added, which suggests that most features are irrelevant and it matched my assumption well. (with only minor fluctuations at layer 10)

To further verify this result, I also applied the backward elimination algorithm. The result of backward elimination is shown in Figure 4. In this algorithm, the search starts from the full feature set containing all 16 features, which gives an accuracy of 70.2%, the same result we obtained earlier when using all 16 features in forward selection.

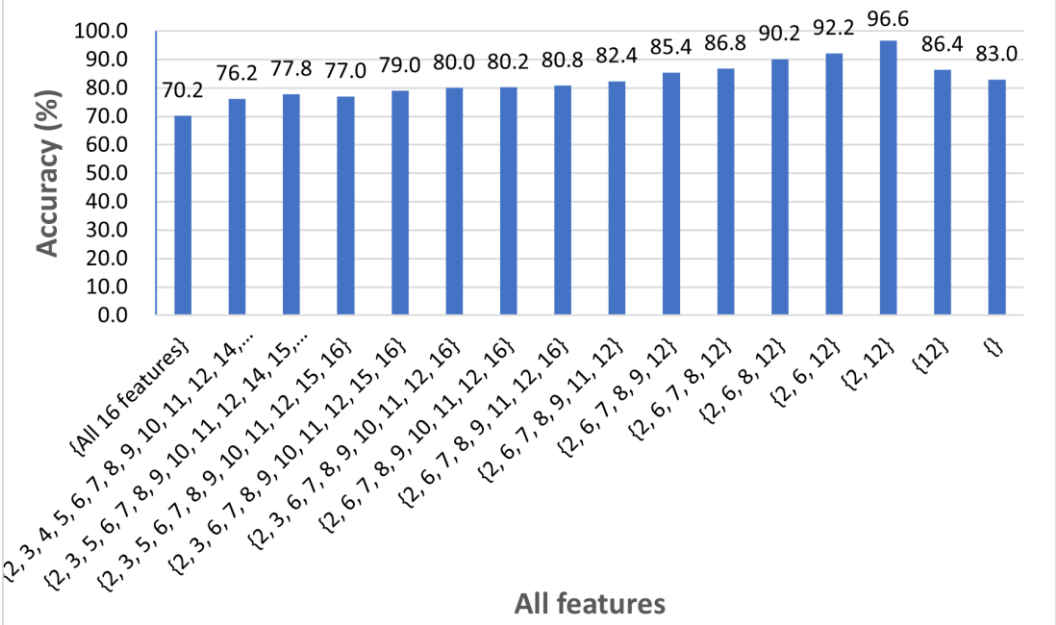


Figure 4: Accuracy changes during backward elimination in small dataset 1

Then the algorithm starts to remove features at each level. With the accuracy gradually increasing, it eventually gets to the peak at feature set {2,12}, with the accuracy of 96.6%. Interestingly, this is the same feature subset result from forward selection. To gain a better understanding of the final results, I visualized a scatter plot in a 2D space, using only feature 2 and feature 12, shown in Figure 5.

From the scatter plot, the two classes appear to form a structure that has the same centroid, similar to “two concentric rings” in the class. As discussed in class, such a pattern is difficult for many other classifiers. For example, a decision tree would

have to create large branches of partitions like steps, and Naive Bayes can produce a quadratic decision boundary[5], but it would still fail here, not to mention the linear classifier.

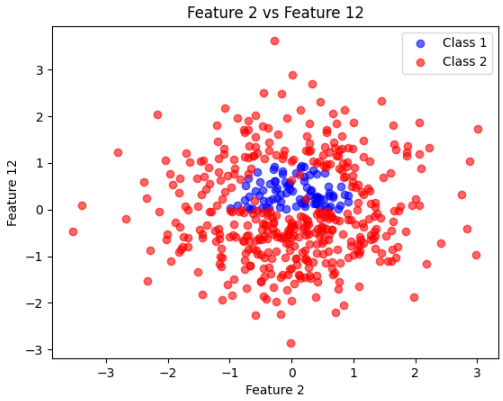


Figure 5: Scatter plot of the small dataset using feature 2 and 12

Finally, based on the results above, we can claim with confidence that: features 2 and 12 are the most informative features that give the best accuracy of 96.6% for this small dataset 1.

Large Dataset Analysis

In the large dataset (large dataset 1), it contains 3000 instances, each belonging to either class 1 or class 2, with 64 continuous features. The search space increases to 2^64 and it is more challenging. The data is also extremely unbalanced. Class 1 has only 559 instances, while class 2 has 2441 instances, and the default rate is $2441/3000 \approx 81.4\%$. Similarly, the leave-one-out validation strategy is applied to the algorithm, getting the accuracy of 68.0% for all 64 features. It again suggests that most of the features seem useless in our selection. With this in mind, I began my feature search.

[5] While our class focuses on quadratic boundaries, it should be noted that under the assumption of conditional distributions, a Gaussian distribution with unequal covariance matrices can generate oval decision boundary. (Learned from CS171, Machine Learning and data mining, taught by Dr. Asif)

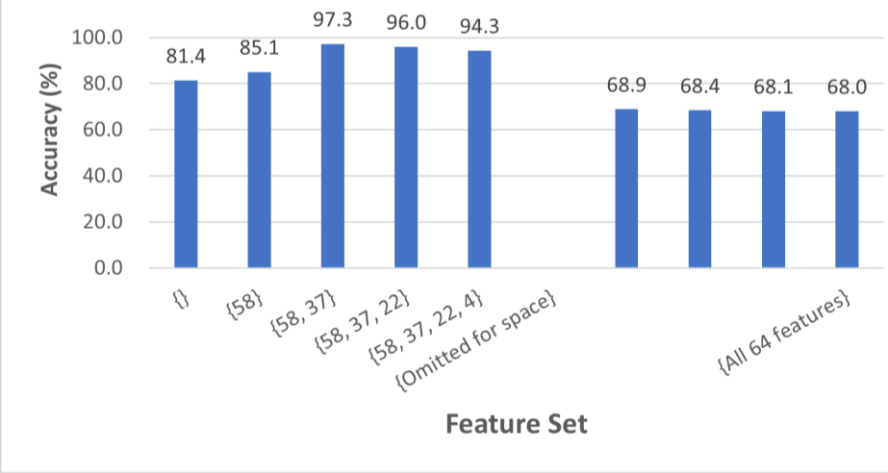


Figure 6: Accuracy changes during forward selection in large dataset 1

As shown in Figure 6, in the forward selection, the search starts from the empty feature set with 81.4% default rate. At the first level, adding feature {58} improved the most accuracy to 85.1%. At the second level, adding feature {37} greatly increased the accuracy to 97.3%. However, when additional features are added, the performance is getting worse. For example, adding feature 22 reduced the accuracy to 96.0%, and adding feature 4 decreased it to 94.3%. It perfectly matched my assumption.

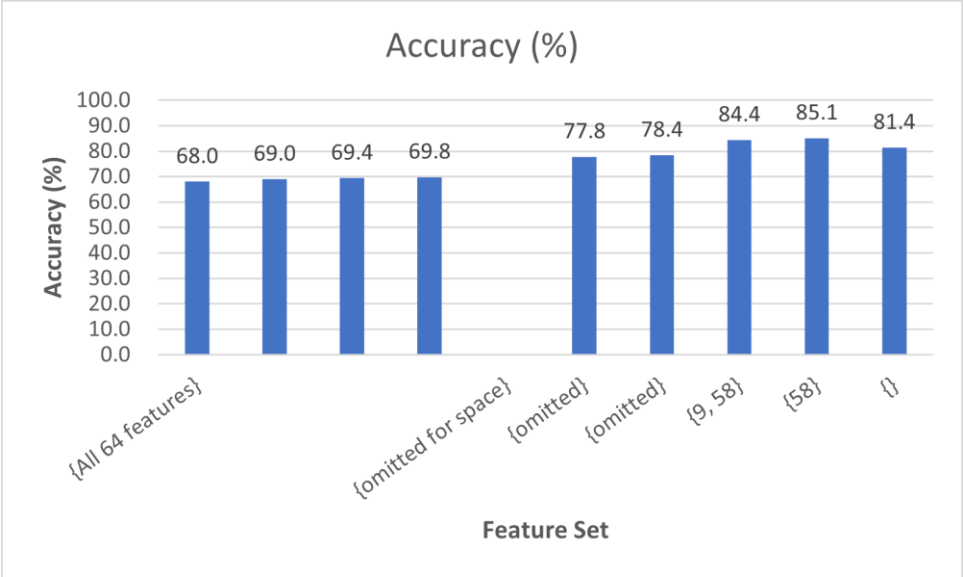


Figure 7: Accuracy changes during backward elimination in large dataset 1

The backward elimination algorithm was also applied. The result is shown in Figure 7. As features are gradually removed, the accuracy steadily improves. Near the end

of the search, when only few features remain, the performance increases quickly. Particularly, the subset {9, 58} achieves 84.4%, and the feature subset {58} achieves an accuracy of 85.1%.

The two algorithms produce different results on this dataset. In forward selection, the subset {58, 37} achieves the highest accuracy of 97.3%. One possible reason is that forward selection builds the feature set step by step, which gives more combinations of features when there are only few features. In contrast, backward elimination process is more restricted to some extent. Once an important combination of the subset is removed, it is impossible to recover back the feature. This is also a limitation of hill-climbing algorithms, as illustrated in Figure 8. As shown in Figure 9, the scatter plot of Feature 37 and 58 also reveals a concentric structure. This visualization explains why these two features together achieve 97.27% accuracy.

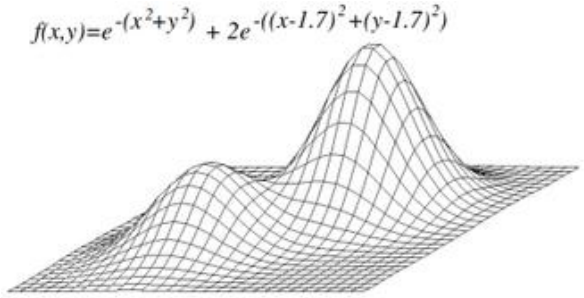


Figure 8: A surface with two local maxima. [6]

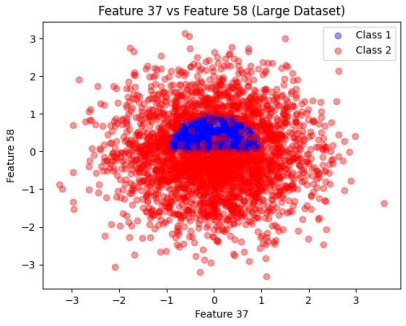


Figure 9: Scatter plot of the large dataset using feature 58 and 37

Finally, based on the forward selection result, the feature subset {58, 37} is the most informative and has the best performance, achieving the top accuracy of 97.3% on the large dataset.

Computational Effort

As we know, a good algorithm should both guarantee to find a reasonable solution and be efficient in computational effort. I implemented the search in Python and ran all experiments on the UCR BOCE CPU clusters[7].

	Small Dataset	Large Dataset

6 If a hill-climber begins in a poor location, it may converge to the lower maximum. The image is from hill climbing algorithm at Wikipedia: https://en.wikipedia.org/wiki/Hill_climbing

7 Bourns College of Engineering CPU Clusters: The high-performance computing (HPC) environment provided by UCR: https://systems.engr.ucr.edu/profile

Forward Selection	3.4 seconds	1763.0 seconds
Backward Elimination	3.7 seconds	3370.1 seconds

Table 1: Computational time comparison

As shown in Table 1, the small dataset finished the search in just a few seconds for both algorithms. When it comes to the large dataset, the computation time cost has a dramatic increase. Forward selection took about 29 minutes while backward elimination took nearly 56 minutes. It should be noted that forward selection is faster compared to the backward elimination. This is because it starts from an empty set and evaluates fewer features at the early stages. This difference becomes more obvious as the number of features increases. We can also infer that for both approaches, if the scale of dataset features continues to increase, the computational time will explode.

Reflection and Future Improvement

One improvement I made during this project was changing the for loop with numpy[8] vectorized operations when calculating the Euclidean distance. Using naïve for loop is extremely slow, especially when running backward elimination, where even a single level could not be completed after computing for one night. I immediately realized that it was inefficient and not suitable for this task. After switching to a vectorized implementation, I was able to complete the full search in about 56 minutes.

Additionally, as mentioned in the last part, another limitation of this project is that both algorithms are greedy algorithms, which may get stuck in local optima. For the future improvement, I would consider applying Simulated Annealing[9]. *By cooling the temperature[10] slowly, the global maximum is found.* This would be particularly useful for our large dataset case (with 64 features) and might potentially lead to better feature subsets compared to greedy methods.

[8] Vector calculation: https://numpy.org/doc/stable/reference/generated/numpy.exp.html; https://numpy.org/doc/stable/user/basics.broadcasting.html

[9] From Dr. Keogh's lecture slide 5_Optmizing_search: https://www.dropbox.com/scl/fo/blbkjaf1eyl94lij5wl2b/AAvNKn0YrnaX0oGPQn7ueFo?dl=0&e=2&preview=5__Optimizing+Search.pptx&rlkey=alq2gb2ftsw73hcar4lk897r0

[10] The "temperature" is a parameter that controls the probability of accepting worse solutions during the search process. The sentence is cited from the Wikipedia: https://en.wikipedia.org/wiki/Simulated_annealing

Conclusion

Considering the results of the two algorithms and the comparisons between them: forward selection and backward elimination, some conclusions can be made:

- For the small dataset 1, both algorithms converged to the same result of feature subset {2, 12}, achieving an accuracy of 96.6%. From the scatter plot, we can conclude features 2 and 12 are truly the most informative features.
- For the large dataset 1, both algorithms produced different results. Forward selection found the subset {58, 37} with 97.3% accuracy, while backward elimination only found {58} with the accuracy of 85.1%. In this case, forward selection performs better and features 58 and 37 are the most informative features.
- Also, using leave-one-out strategy with all features actually performs worse than the default rate. This strongly indicates that irrelevant features can hurt the performance of KNN classifier.

Supporting Information and Algorithm Trace

All code, output logs, and figures are available on GitHub, with ten days commit history: https://github.com/cyy-07/UCR-CS170-Project2_ML

The structure was inspired by Dr. Eamonn's pseudocode, and a website: https://www.geeksforgeeks.org/machine-learning/forward-feature-selection-in-machine-learning/.
Below, I present a single trace of my algorithm, showing Forward Selection on the small dataset. The first level and the last level are retained, and the middle levels are deleted for space:

```
Welcome to Yiyang Chen's ML project2!
This project is a feature selection algorithm, using Nearest Neighbor as the classifier!
Please pick a dataset you want:
1) Small dataset
2) Large dataset
1
This dataset has 16 features(not including class label) and 500 instances.

Please pick a feature selection algorithm you want:
1) Forward Selection
2) Backward Elimination
1
This dataset has 16 features(not including class label) and 500 instances.
Running nearest neighbor with all 16 features, using "leave-one-out" evaluation, I get an accuracy of 70.2%
Beginning search. The current feature set is empty.
Using feature(s) [1] accuracy is 72.20%
Using feature(s) [2] accuracy is 72.60%
Using feature(s) [3] accuracy is 71.00%
Using feature(s) [4] accuracy is 70.40%
Using feature(s) [5] accuracy is 73.60%
Using feature(s) [6] accuracy is 69.20%
Using feature(s) [7] accuracy is 73.40%
Using feature(s) [8] accuracy is 71.80%
Using feature(s) [9] accuracy is 73.40%
Using feature(s) [10] accuracy is 75.20%
Using feature(s) [11] accuracy is 69.60%
Using feature(s) [12] accuracy is 86.40%
Using feature(s) [13] accuracy is 70.00%
Using feature(s) [14] accuracy is 71.00%
Using feature(s) [15] accuracy is 71.40%
Using feature(s) [16] accuracy is 67.40%
Feature set [12] was best, accuracy is 86.40%
Using feature(s) [12, 1] accuracy is 83.60%
Using feature(s) [12, 2] accuracy is 96.60%
Using feature(s) [12, 3] accuracy is 83.80%
Using feature(s) [12, 4] accuracy is 84.80%
Using feature(s) [12, 5] accuracy is 87.20%
Using feature(s) [12, 6] accuracy is 85.00%
Using feature(s) [12, 7] accuracy is 83.40%
Using feature(s) [12, 8] accuracy is 84.40%
Using feature(s) [12, 9] accuracy is 85.40%
Using feature(s) [12, 10] accuracy is 83.80%
Using feature(s) [12, 11] accuracy is 84.40%
Using feature(s) [12, 13] accuracy is 83.80%
Using feature(s) [12, 14] accuracy is 85.40%
Using feature(s) [12, 15] accuracy is 85.20%
Using feature(s) [12, 16] accuracy is 83.00%
Feature set [12, 2] was best, accuracy is 96.60%
```

{Dear Professor: Here I deleted most of the trace to save space}

Warning! At this level, the best accuracy is worse than the best overall accuracy! Continue search to see if we can find a better feature subset.(Avoiding local maximum!)
Using feature(s) [12, 2, 1, 4, 3, 5, 10, 7, 13, 9, 14, 6, 16, 11, 8] accuracy is 71.20%
Using feature(s) [12, 2, 1, 4, 3, 5, 10, 7, 13, 9, 14, 6, 16, 11, 15] accuracy is 69.80%
Feature set [12, 2, 1, 4, 3, 5, 10, 7, 13, 9, 14, 6, 16, 11, 8] was best, accuracy is 71.20%
Warning! At this level, the best accuracy is worse than the best overall accuracy! Continue search to see if we can find a better feature subset.(Avoiding local maximum!)
Using feature(s) [12, 2, 1, 4, 3, 5, 10, 7, 13, 9, 14, 6, 16, 11, 8, 15] accuracy is 70.20%
Feature set [12, 2, 1, 4, 3, 5, 10, 7, 13, 9, 14, 6, 16, 11, 8, 15] was best, accuracy is 70.20%
Warning! At this level, the best accuracy is worse than the best overall accuracy! Continue search to see if we can find a better feature subset.(Avoiding local maximum!)
Finished search! The best feature subset is [12, 2], which has an accuracy of 96.60%
Total search time: 3.4 seconds

Code: Below is my code for this project, and all code, output logs, and figures are available on GitHub: https://github.com/cyy-07/UCR-CS170-Project2_ML

```
import numpy as np
import time
#calculate distance
def Euclidean_distance(point1, point2):
    total = 0
    for i in range(len(point1)):
        diff = point1[i] - point2[i]
        total += diff** 2
    distance = total ** 0.5
    return distance
# tsetpoint_a = [1, 2, 3]
# tsetpoint_b = [4, 6, 3]
# print(Euclidean_distance(tsetpoint_a, tsetpoint_b))
#Debug: for loop is too slow, we use numpy to calculate the distance.
#Compare distance between the point and all other points in the dataset, return the class label as the nearest neighbor
def nn_classify(dataset, feature_set):
    correct_classification_number = 0
    data=dataset[:, feature_set]
    labels=dataset[:, 0]
    for i in range (len(dataset)):
        current_point = data[i]
        diff=data - current_point
        distance=np.sqrt(np.sum(diff**2, axis=1))
        distance[i] = float('inf')  # Set the distance to itself to infinity, in order to avoid being selected as the nearest neighbor
        nn_label = labels[np.argmin(distance)]
        if nn_label == labels[i]:
            correct_classification_number += 1
    accuracy = correct_classification_number / len(dataset)

    return accuracy

def forward_selection(dataset, features_num):
    start = time.time()
    current_feature_set = []
    the_best_overall_accuracy = 0
    the_best_overall_feature_set = []
    all_features = []                #Here we initialize a list to store all the features, using a for loop to add all the features to this list.
    for f in range(1, features_num + 1):
        all_features.append(f)
    overall_accuracy = nn_classify(dataset, all_features)
    print(f'Running nearest neighbor with all {features_num} features, using "leave-one-out" evaluation, I get an accuracy of {overall_accuracy * 100:.1f}%')
    print("Beginning search. The current feature set is empty.")
    #For each level, we will add a new feature to the current feature set,
    #and we compute each feature's accuracy.
    #Finally, we will pick the feature with the highest accuracy and add it to the current feature set.
    for i in range (1,features_num+1):
        best_accuracy_this_level = 0
        best_feature_this_level = None
        for j in range (1,features_num+1):     #note that we start from one because the feature index begins from 1!
            if j not in current_feature_set:     #consider only the features that are not in the current feature set
                current_feature_set.append(j)            #add the feature to the current feature set temporarily
                accuracy = nn_classify(dataset, current_feature_set)
                print(f"Using feature(s) {current_feature_set} accuracy is {accuracy * 100:.2f}%")
                current_feature_set.remove(j)            #debug: remove the feature from the current (temporary) feature set.
                if accuracy > best_accuracy_this_level:      #if the accuracy is better than the best accuracy we have now, update the best accuracy.
                    best_feature_this_level = j
                    best_accuracy_this_level = accuracy
        #After we finish testing all the features that are not in the current feature set, we add the best feature of this level to current feature set.
        current_feature_set.append(best_feature_this_level)
        print(f"Feature set {current_feature_set} was best, accuracy is {best_accuracy_this_level * 100:.2f}%")
        if best_accuracy_this_level > the_best_overall_accuracy:
            the_best_overall_accuracy = best_accuracy_this_level
            the_best_overall_feature_set = current_feature_set.copy()
```

```
    else:
        print("Warning! At this level, the best accuracy is worse than the best overall accuracy! Continue search to see if we can find a better feature subset.(Avoiding local maximum!)")
    print(f"Finished search! The best feature subset is {the_best_overall_feature_set}, which has an accuracy of {the_best_overall_accuracy * 100:.2f}%")
    print(f"Total search time: {time.time() - start:.1f} seconds")          #Here we calculate the total search time.

def backward_elimination(dataset, features_num):
    #Start with everything, and remove a feature at each level.
    start = time.time()
    current_feature_set = []
    for i in range(1, features_num + 1):
        current_feature_set.append(i)      #Different from forward selection, we start with the full feature set in backward elimination here.
    the_best_overall_accuracy = 0
    the_best_overall_feature_set = current_feature_set.copy()

    overall_accuracy = nn_classify(dataset, current_feature_set)
    print(f'Running nearest neighbor with all {features_num} features, using "leave-one-out" evaluation, I get an accuracy of {overall_accuracy * 100:.1f}%')
    print("Beginning search. The current feature set is full.")
    for i in range(1, features_num):
        best_accuracy_this_level = 0
        best_feature_to_remove = None
        for j in range(1, features_num + 1):
            #Try removing each feature from the current feature set, then calculate the accuracy, after removing that feature.
            if j in current_feature_set:
                current_feature_set.remove(j)            #remove the feature from the current feature set temporarily.
                accuracy = nn_classify(dataset, current_feature_set)
                print(f"Using feature(s) {current_feature_set} accuracy is {accuracy * 100:.2f}%")
                current_feature_set.append(j)            #put feature j back to the current feature set.
                if accuracy > best_accuracy_this_level:
                    best_feature_to_remove = j
                    best_accuracy_this_level = accuracy
        current_feature_set.remove(best_feature_to_remove)  #After we finish testing all the features in the current feature set, we remove the best feature of this level from current feature set.
        print(f"Feature set {current_feature_set} was best, accuracy is {best_accuracy_this_level * 100:.2f}%")
        if best_accuracy_this_level > the_best_overall_accuracy:
            the_best_overall_accuracy = best_accuracy_this_level
            the_best_overall_feature_set = current_feature_set.copy()
        else:
            print("Warning! At this level, the best accuracy is worse than the best overall accuracy! Continue search to see if we can find a better feature subset.(Avoiding local maximum!)")
    print(f"Finished search! The best feature subset is {the_best_overall_feature_set}, which has an accuracy of {the_best_overall_accuracy * 100:.2f}%")
    print(f"Total search time: {time.time() - start:.1f} seconds")

print("Welcome to Yiyang Chen's ML project2!")
print("This project is a feature selection algorithm, using Nearest Neighbor as the classifier!")

print("\nPlease pick a dataset you want:")
print("1) Small dataset")
print("2) Large dataset")

while True:
    dataset_choose = input()
    if dataset_choose == '1':
        dataset = np.loadtxt('CS170_Small_DataSet__1.txt')#Note that Dr.Eamonn suggested me to use small and large dataset 1 because my name was not in the list.
        break
    elif dataset_choose == '2':
        dataset = np.loadtxt('CS170_Large_DataSet__1.txt')
        break
    else:
        print("Invalid input, please choose 1 or 2 again.")

features_num = dataset.shape[1] - 1
instance_num = dataset.shape[0]
print(f"This dataset has {features_num} features(not including class label) and {instance_num} instances.")

while True:
    print("\nPlease pick a feature selection algorithm you want:")
    print("1) Forward Selection")
    print("2) Backward Elimination")
```

```
    algorithm_choose = input()
    if algorithm_choose == "1" or algorithm_choose == "2":
        break
    else:
        print("Invalid input, please choose 1 or 2 again.")
if algorithm_choose == "1":
    forward_selection(dataset, features_num)
elif algorithm_choose == "2":
    backward_elimination(dataset, features_num)
```

Extra Credit for Project 2: Banknote dataset

I chose to work with the *banknote authentication dataset*[11] from the UCI repository. I found it interesting, and the features were based on image processing, which also interested me. In the extra credit task, we are required to classify whether a banknote is genuine (class 1) or forged (class 2). We are given 4 features: Variance, Skewness, Kurtosis, and Entropy, and the dataset has 1372 instances in total.

In forward selection algorithm, starting from the empty set, the default rate is 55.5%. Adding feature 1 "Variance" alone dramatically improved the accuracy to 79.0%, as shown in Figure 10. At the second layer, feature 2 Skewness further increased the accuracy to 94.2%. When Feature 3 Kurtosis was included, the accuracy reached the top at 99.93%! This observation is surprisingly high for my nearest neighbor classifier.

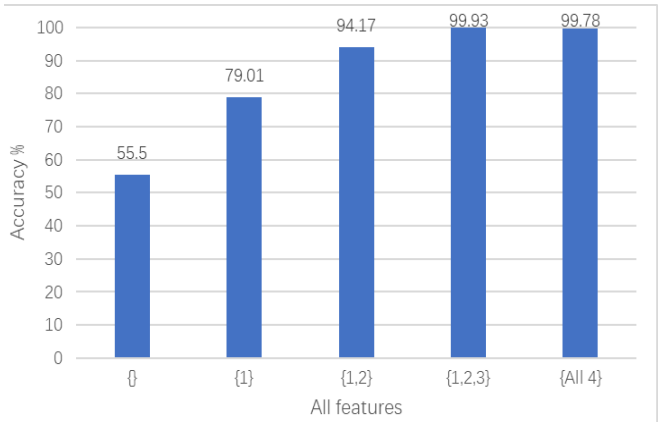


Figure 10: Accuracy changes during forward selection in banknote dataset

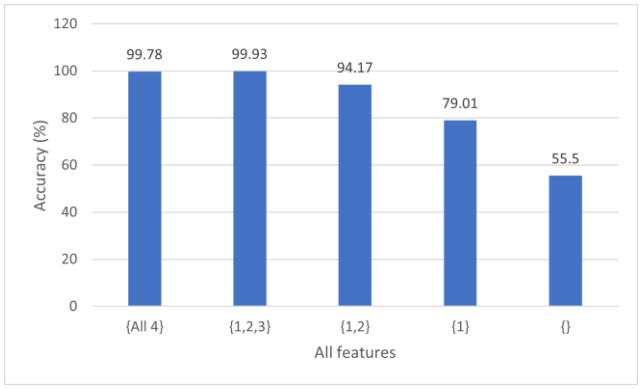


Figure 11: Accuracy changes during backward elimination in banknote dataset

From the figures we can observe that both algorithms generated perfectly mirrored results, with selecting the same optimal subset {1, 2, 3}. One possible reason is that the dataset contains only four features, which makes the search space very small. I also found some similar work using the same dataset. According to a related study[12], Decision Tree achieved 99.27% accuracy. Our feature selection algorithm with only 3 features achieved 99.93%, which outperforms their best result.

However, the accuracy of 99.93% seems too high for real world problems and the dataset seems too 'clean'. The images might be noisier, the printing quality might

[11] The download link to the dataset: archive.ics.uci.edu/ml/machine-learning-databases/00267/data_banknote_authentication.txt

[12] Comparing the accuracy of various Classification Algorithms for Banknote Authentication, GitHub, https://github.com/tdishant/Banknote-Authentication

vary, and the features might not be as cleanly separable. Therefore, I suspect when it comes to application, the actual accuracy might be lower.